

IT-SIMPLERA INSTITUTE

Empowering Future Cloud Professionals

WEEK 05 ASSIGNMENT REPORT

AWS Database Services — Amazon RDS (MySQL) & DynamoDB Overview

Field	Details
Prepared By	Zohaib Ali (Cloud Computing Instructor)
Submitted By	Jaweria Mushtaq
Assigned To	Cloud Computing Interns
Week	Week 05
Topic	AWS Database Services (Amazon RDS, MySQL, DynamoDB)
Submission Deadline	25 / 07 / 2026

1. Assignment Overview

This report documents the hands-on implementation of Database Services on AWS, completed as part of the Week 05 assignment at IT-Simplera Institute. It covers relational database concepts, Amazon RDS deployment with MySQL, connecting to the database using MySQL Workbench, creating and managing tables with SQL, an overview of Amazon DynamoDB, and database backup and recovery using RDS snapshots. All steps below were performed live on AWS and verified with screenshots at each stage.

2. Theory: AWS Database Services

2.1 Relational Databases

A relational database organizes data into tables made up of rows and columns, where relationships between tables are defined through keys (primary keys and foreign keys). Data integrity is enforced through a fixed schema, and data is queried and manipulated using Structured Query Language (SQL). Relational databases are well suited for structured data with clear relationships, such as customer records, orders, and inventory.

2.2 Amazon RDS (Relational Database Service)

Amazon RDS is a fully managed relational database service that automates time-consuming administrative tasks such as hardware provisioning, database setup, patching, and backups. It supports several database engines, including MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server. RDS handles routine tasks like automated backups, software patching, and failover, allowing developers to focus on application logic rather than database administration.

2.3 MySQL Basics

MySQL is one of the most widely used open-source relational database management systems. It uses standard SQL for defining and manipulating data through statements such as CREATE TABLE, INSERT, SELECT, UPDATE, and DELETE. When hosted on Amazon RDS, MySQL databases can be accessed remotely using client tools such as MySQL Workbench, the MySQL command-line client, or application drivers.

2.4 DynamoDB Overview

Amazon DynamoDB is a fully managed NoSQL key-value and document database that delivers single-digit millisecond performance at any scale. Unlike RDS, DynamoDB does not require a fixed schema — each item in a table can have a different set of attributes. It is designed for applications that need consistent, low-latency data access, such as gaming leaderboards, session stores, and IoT applications, and it scales automatically to handle unpredictable traffic.

2.5 Database Backup and Recovery

Amazon RDS provides two main mechanisms for protecting data: automated backups and manual snapshots. Automated backups occur daily during a configurable backup window and support point-in-time recovery within the retention period. Manual snapshots are user-initiated and retained until

explicitly deleted, making them useful before major changes such as schema updates or version upgrades. Both types of backups can be used to restore a new RDS instance to a specific point in time

3. Practical Task: Database Services on AWS

The following section documents each step performed in the AWS Management Console and MySQL Workbench, with screenshots captured at every stage as proof of completion.

3.1 Launch an Amazon RDS MySQL Database

The RDS service was opened from the AWS Console, and a new database was created using the following configuration:

- Database creation method: Standard create → Full configuration
- Engine type: MySQL (MySQL Community, version 8.4.9)
- Template: Free tier
- DB instance identifier: itsimplera-week5-db
- Master username: admin
- Instance class: db.t4g.micro (Free tier eligible)
- Public access: Yes (to allow connection from MySQL Workbench)

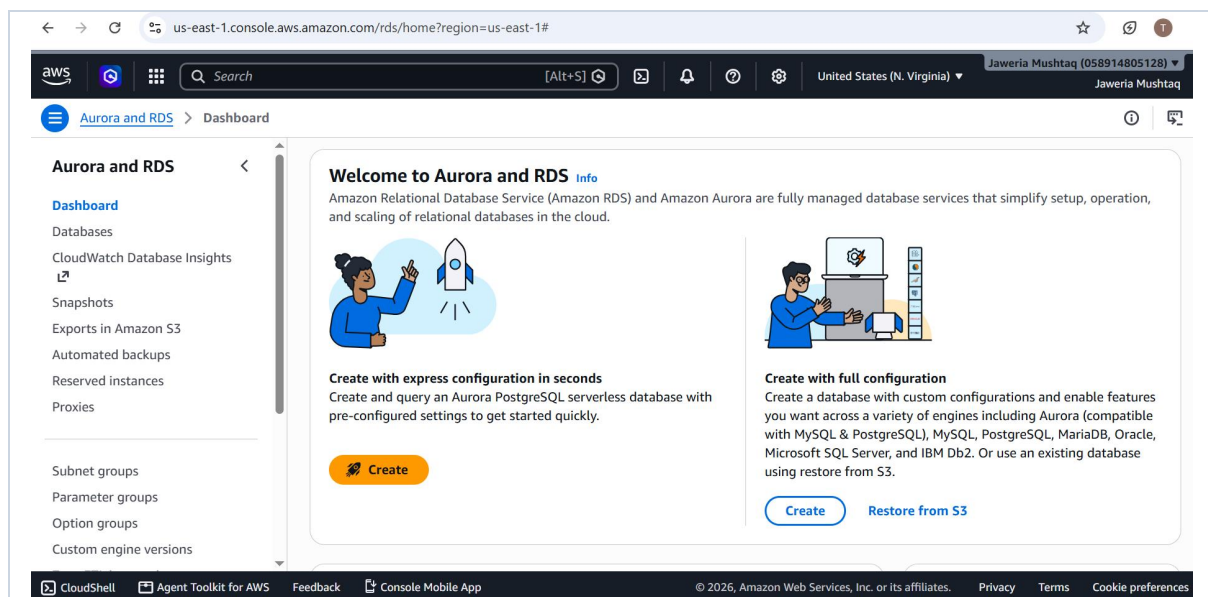


Fig 3.1a — Aurora and RDS Dashboard

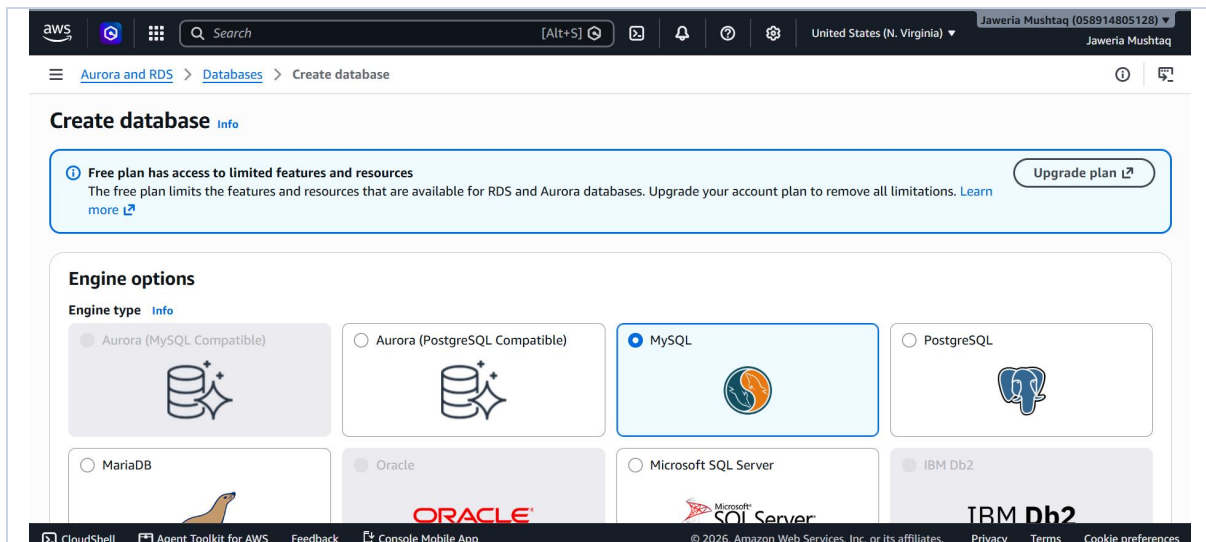


Fig 3.1b — Engine type selected: MySQL

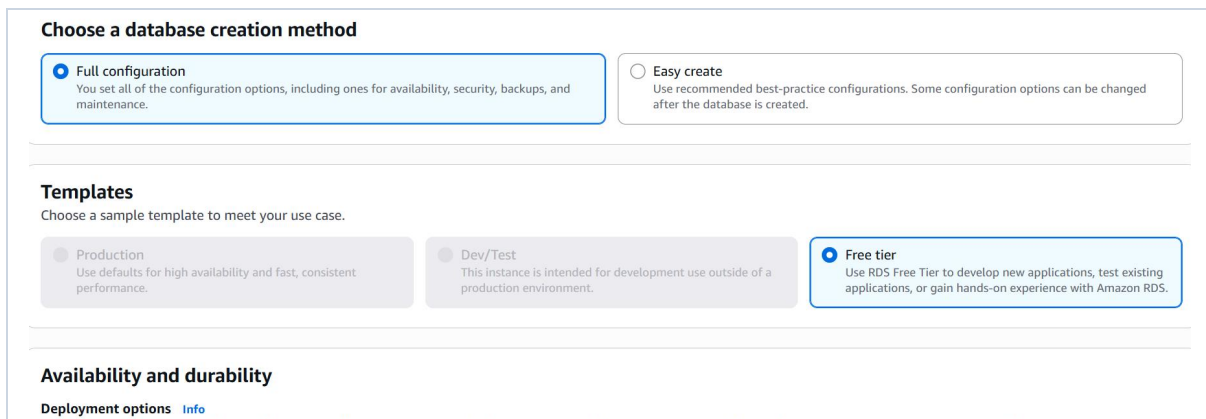


Fig 3.1c — Full configuration method with Free tier template selected

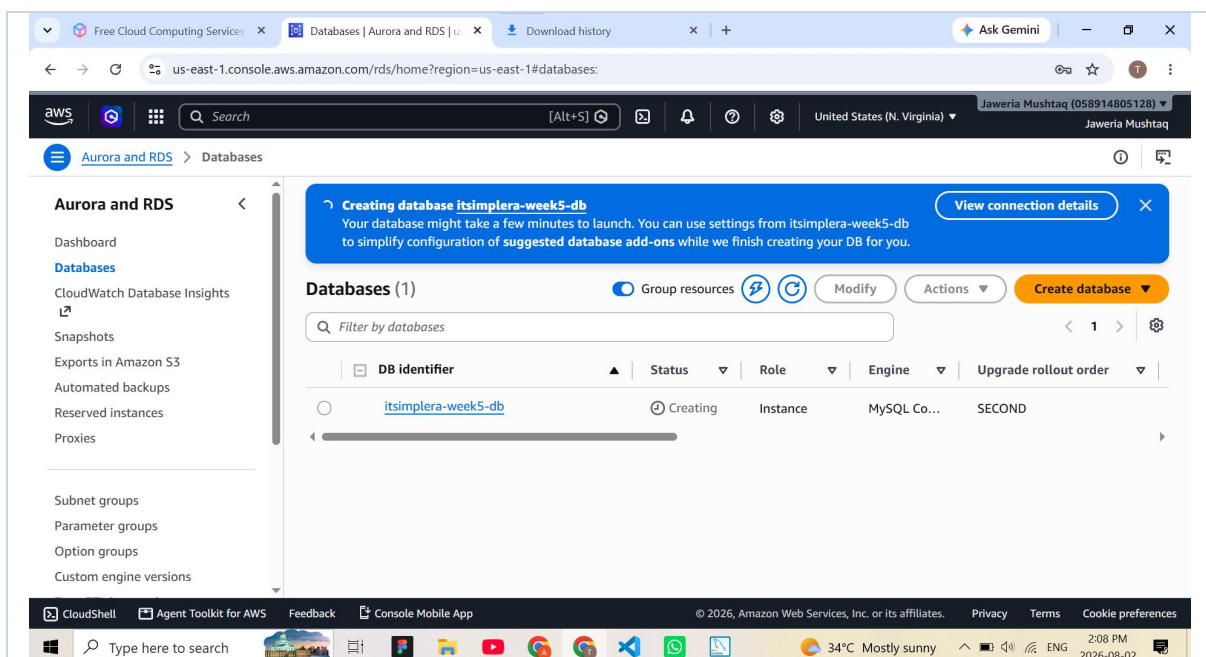


Fig 3.1d — Database creation initiated (status: Creating)

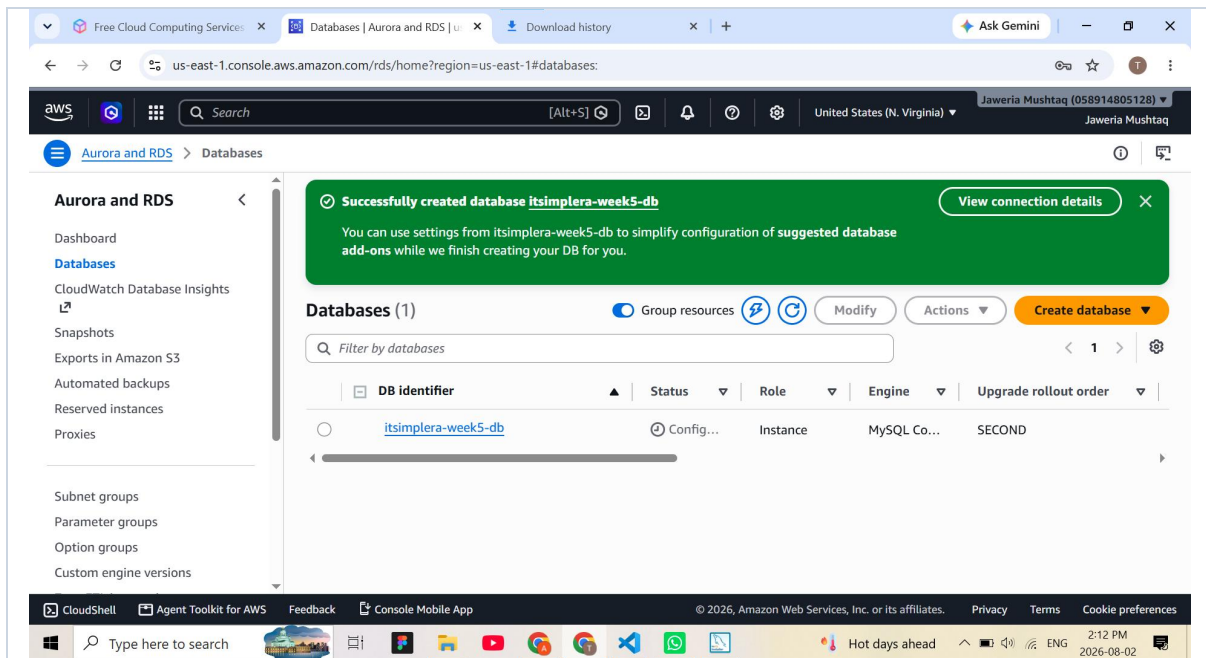


Fig 3.1e — Database successfully created (status: Configuring)

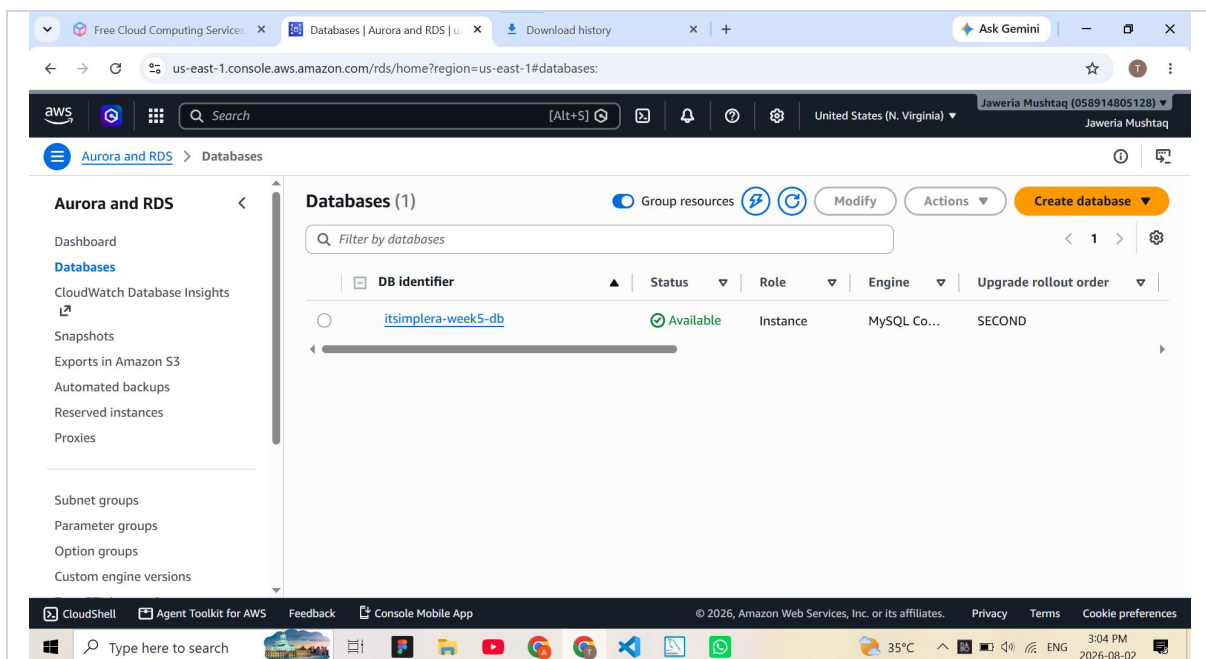


Fig 3.1f — DB instance status changed to Available

3.2 Configure the Database Instance

Once the instance was available, the Connectivity & security tab was reviewed to note the endpoint, port, and VPC security group. The security group's inbound rule was verified to allow MySQL/Aurora traffic (port 3306) from the local IP address.

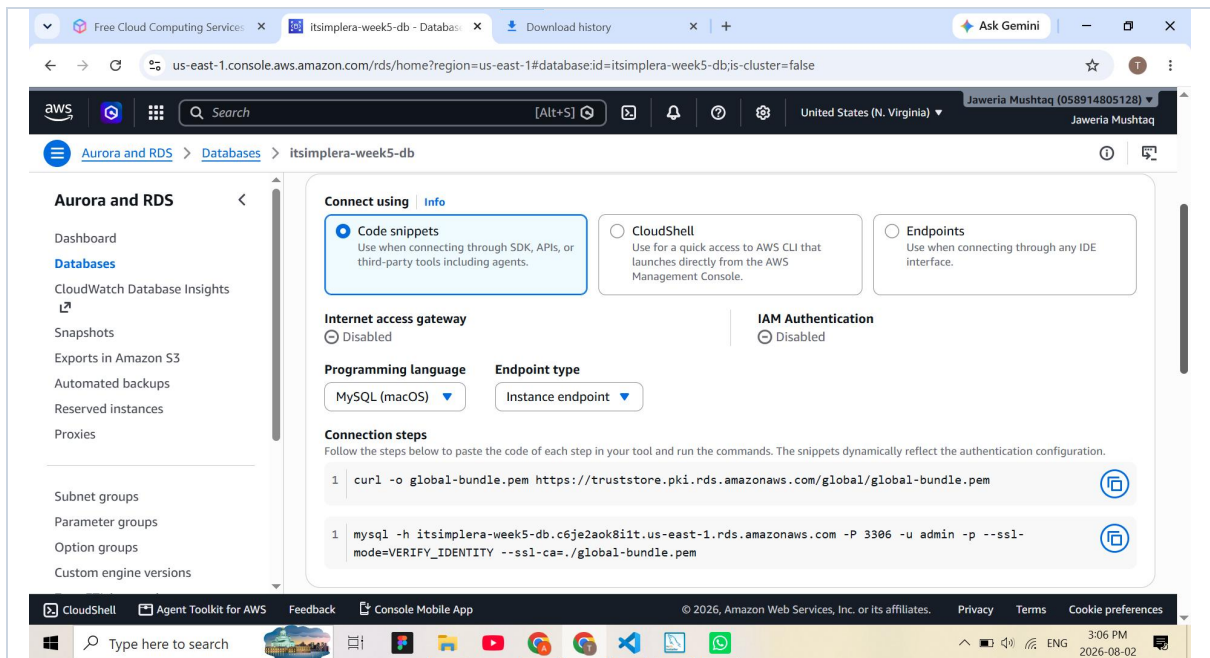


Fig 3.2a — Endpoint and connection details for itsimplera-week5-db

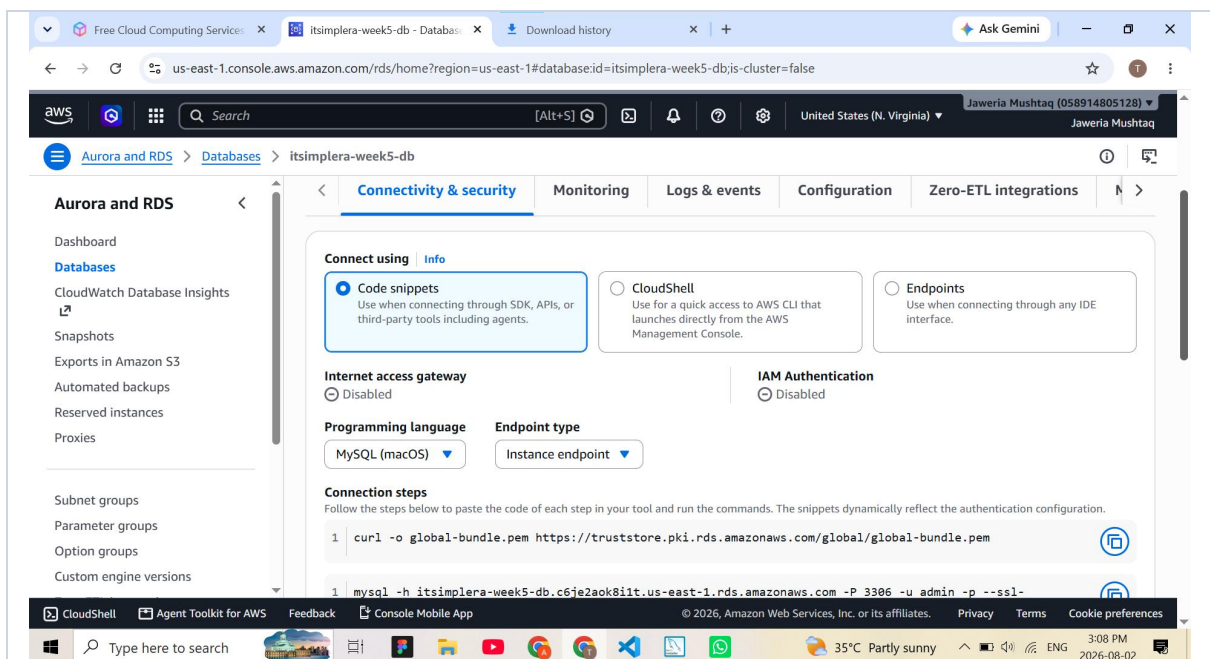


Fig 3.2b — Connectivity & Security tab showing security group rules

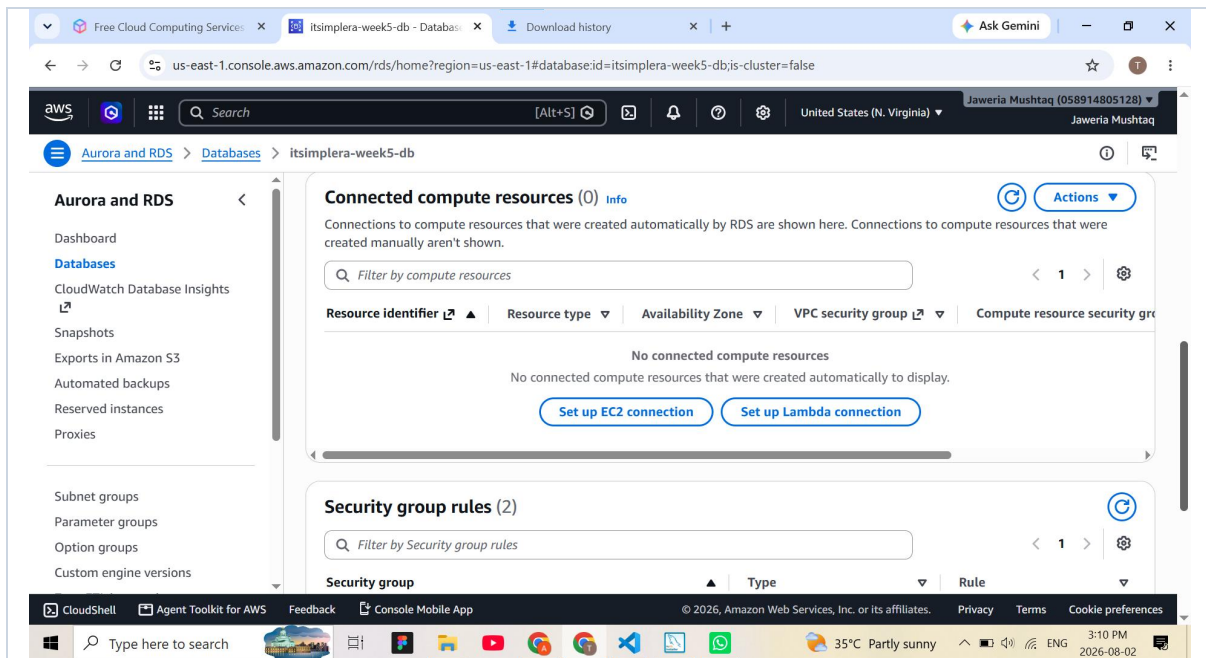


Fig 3.2c — Inbound (port 3306) and outbound security group rules

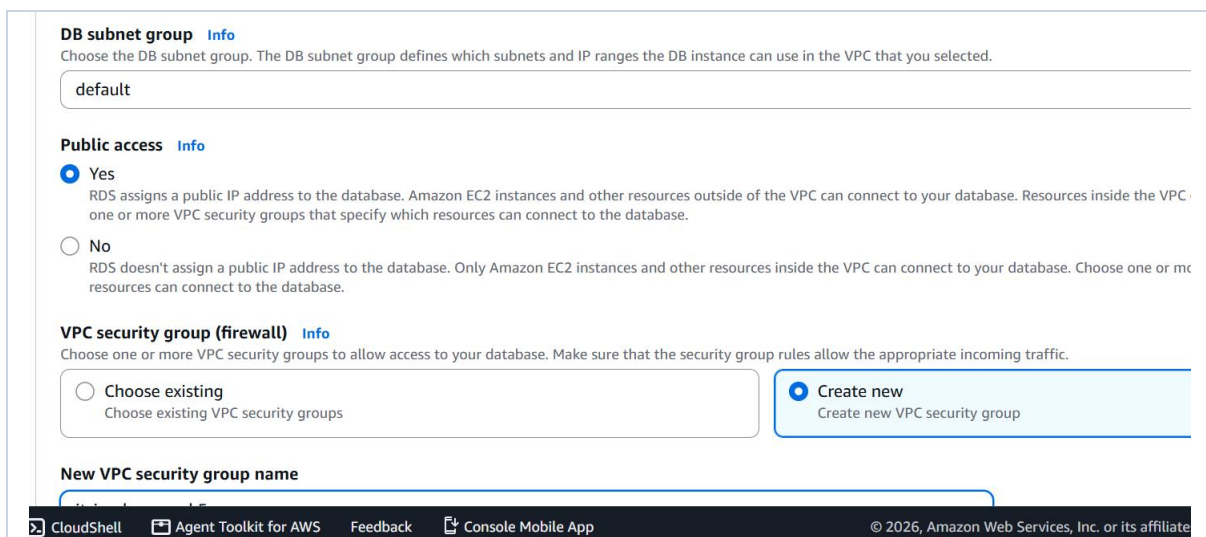


Fig 3.2d — Public access set to Yes during configuration

3.3 Create a New Database

A new database was created inside the RDS MySQL instance using the MySQL Workbench SQL editor with the following statement:

```
CREATE DATABASE itsimplera_week5;
USE itsimplera_week5;
```

3.4 Connect to the Database Using MySQL Workbench

MySQL Workbench was installed and a new connection was configured using the RDS endpoint, port 3306, and the master username. A version-compatibility warning appeared (since the RDS engine version 8.4.9 is newer than Workbench's officially tested versions), which was safely dismissed by selecting "Continue Anyway." The connection test then confirmed a successful SSL-secured connection.

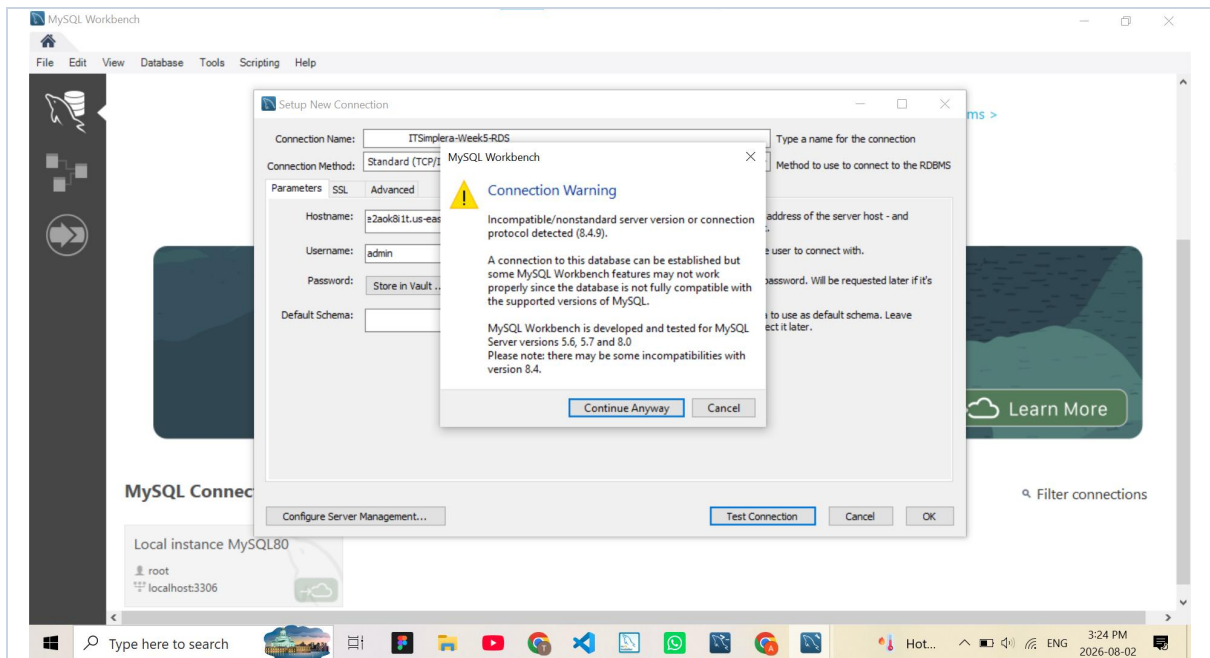


Fig 3.4a — Version compatibility notice (safe to continue)

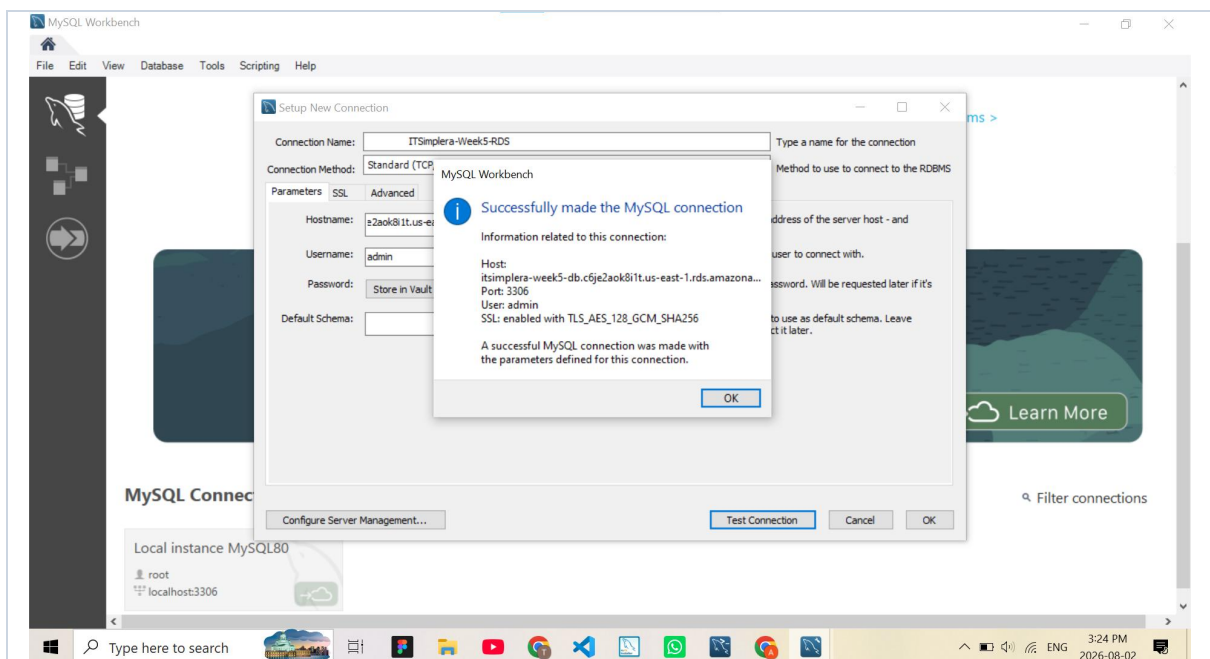


Fig 3.4b — Successfully made the MySQL connection (SSL enabled)

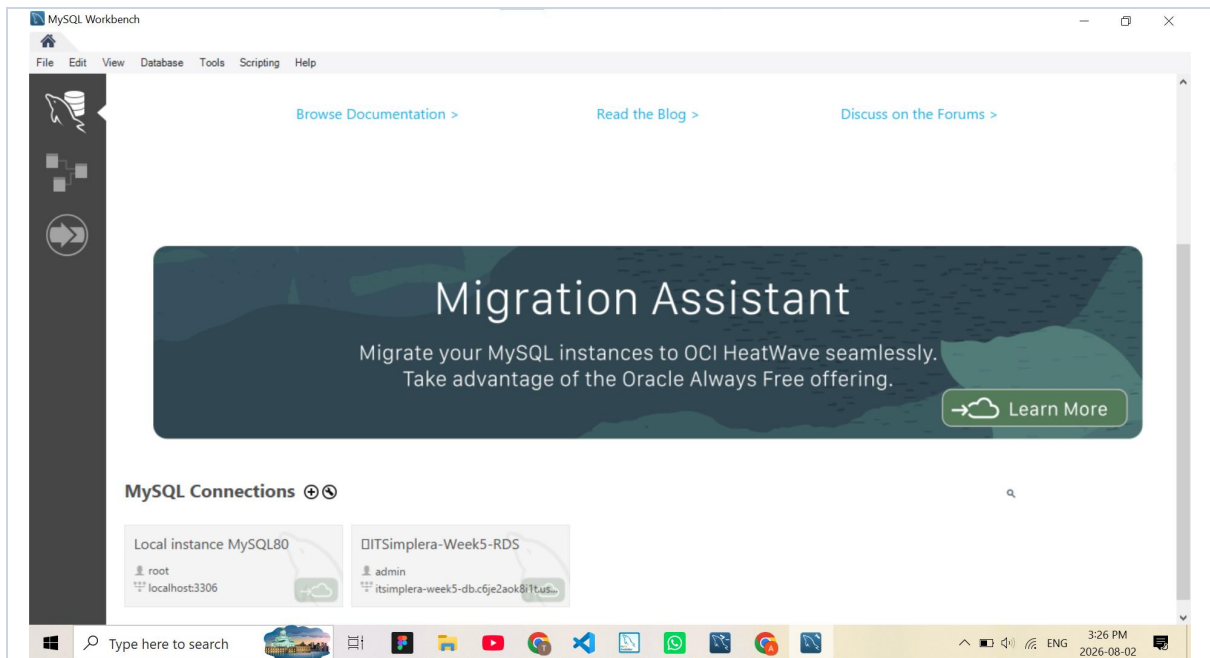


Fig 3.4c — MySQL Workbench home screen showing the new RDS connection tile

3.5 Create and Manage Tables

Inside the itsimplera_week5 database, an interns table was created and populated with sample records using the following SQL statements:

```
CREATE TABLE interns (  
  intern_id INT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  track VARCHAR(50),  
  joined_date DATE  
);  
  
INSERT INTO interns (full_name, email, track, joined_date) VALUES  
(  
  'Ayesha Khan', 'ayesha.khan@example.com', 'Cloud Computing', '2026-07-01'),  
  ('Bilal Ahmed', 'bilal.ahmed@example.com', 'Cloud Computing', '2026-07-01'),  
  ('Sara Malik', 'sara.malik@example.com', 'DevOps', '2026-07-02');
```

Records were then retrieved using SELECT statements:

```
SELECT * FROM interns;  
SELECT full_name, track FROM interns WHERE track = 'Cloud Computing';  
SELECT COUNT(*) AS total_interns FROM interns;
```

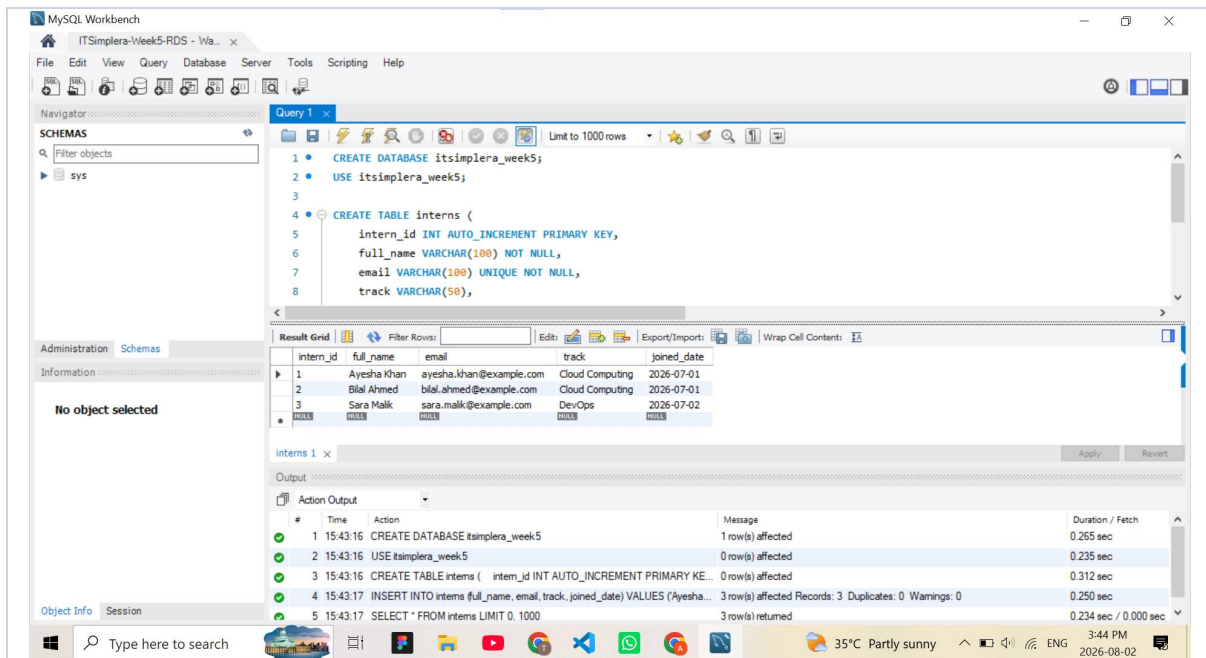


Fig 3.5a — Table created, records inserted, and SELECT * result (3 rows returned)

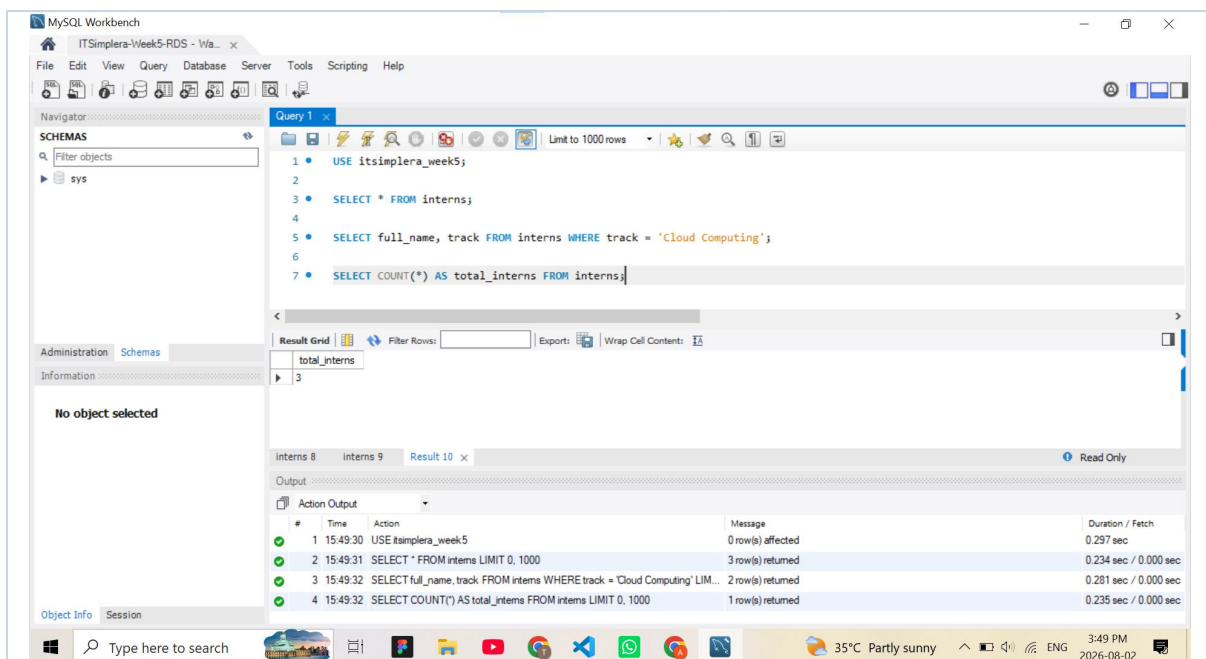


Fig 3.5b — Filtered SELECT and COUNT(*) query results

3.6 Database Backup and Recovery

A manual snapshot named itsimplera-week5-manual-snapshot was taken from the RDS console. The Automated backups section was also reviewed to note the backup retention window.

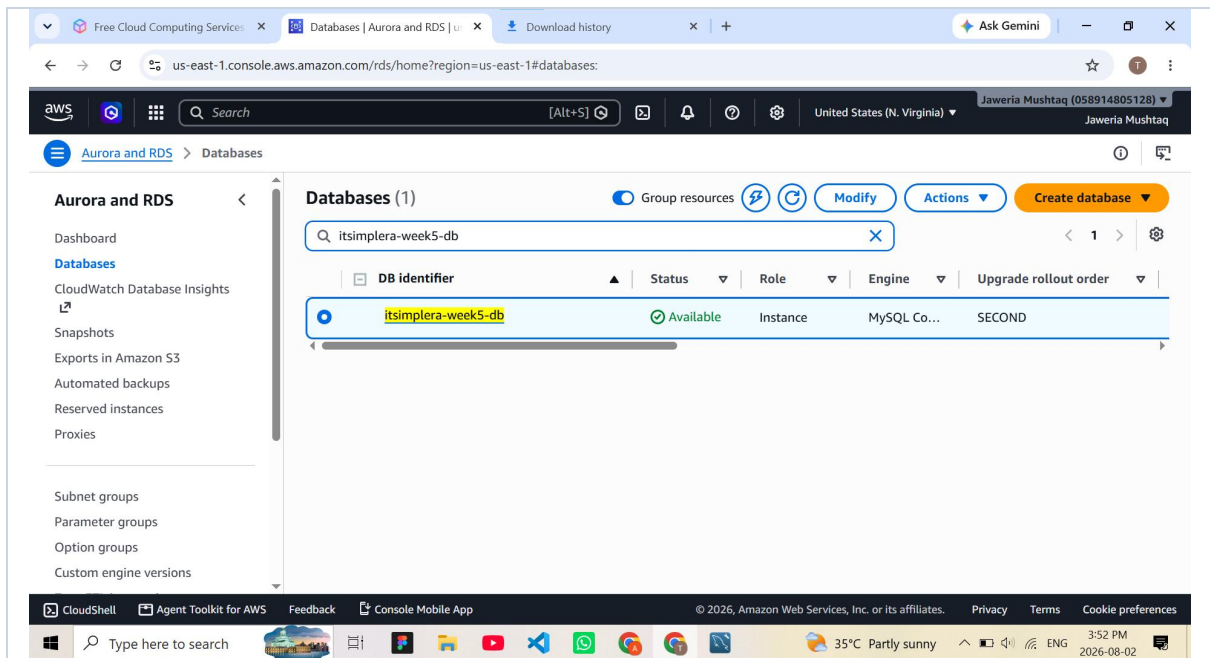


Fig 3.6a — DB instance selected for snapshot action

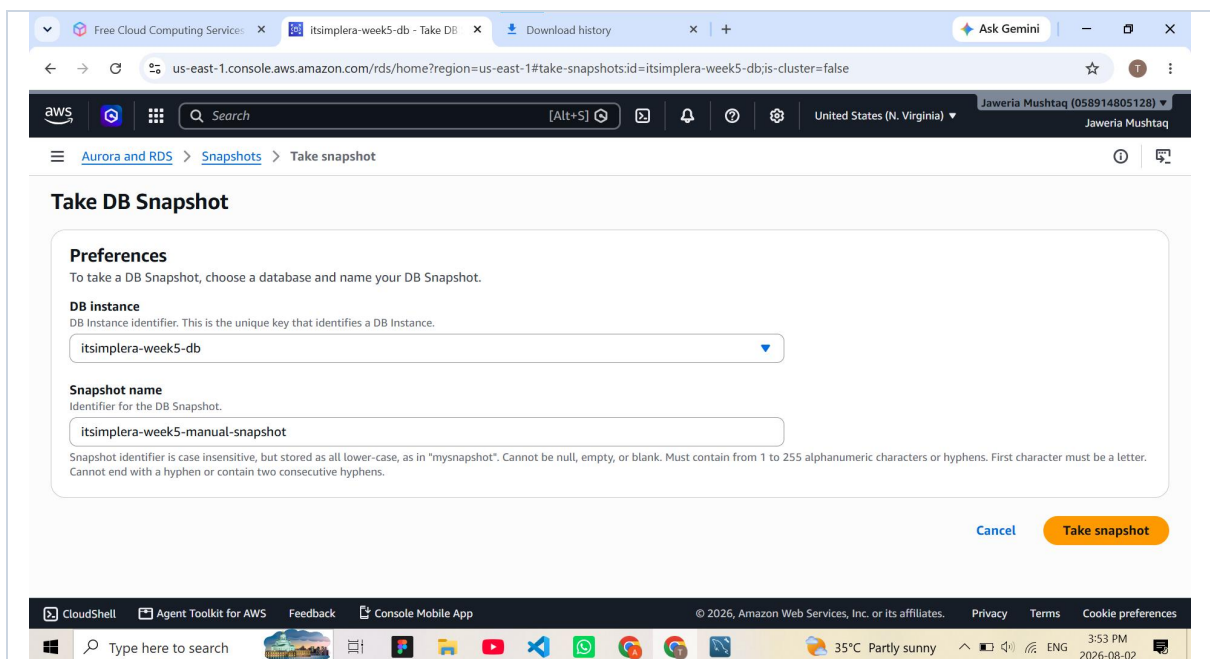


Fig 3.6b — Take DB Snapshot form with snapshot name entered

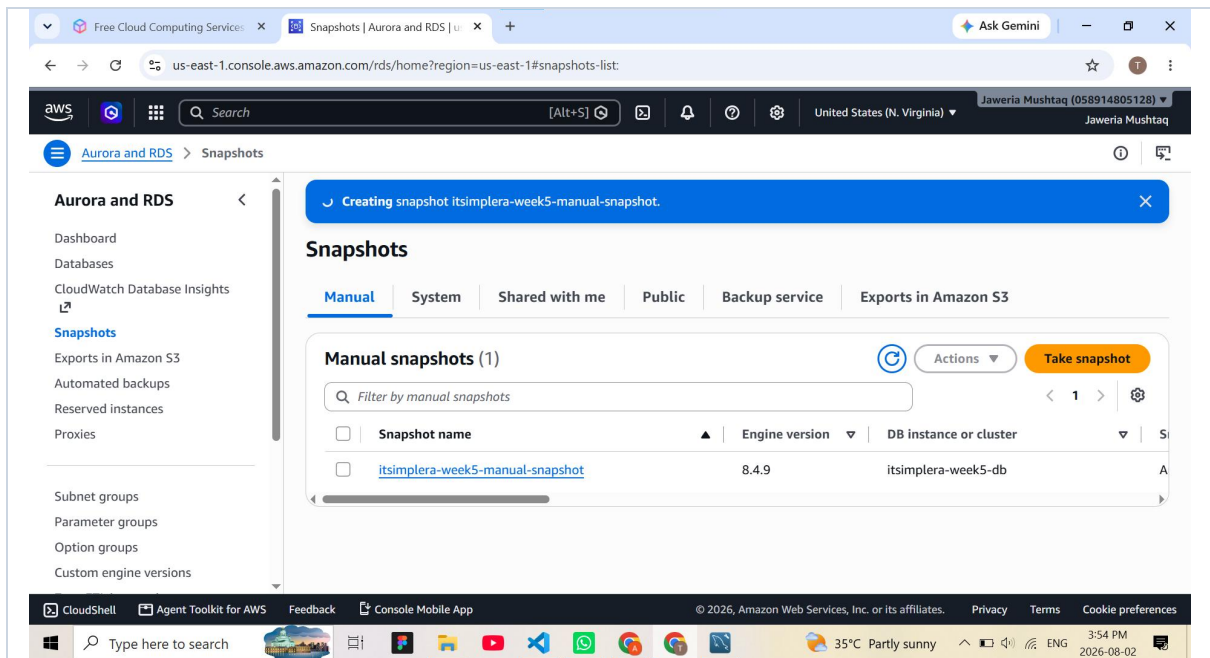


Fig 3.6c — Snapshot creation in progress

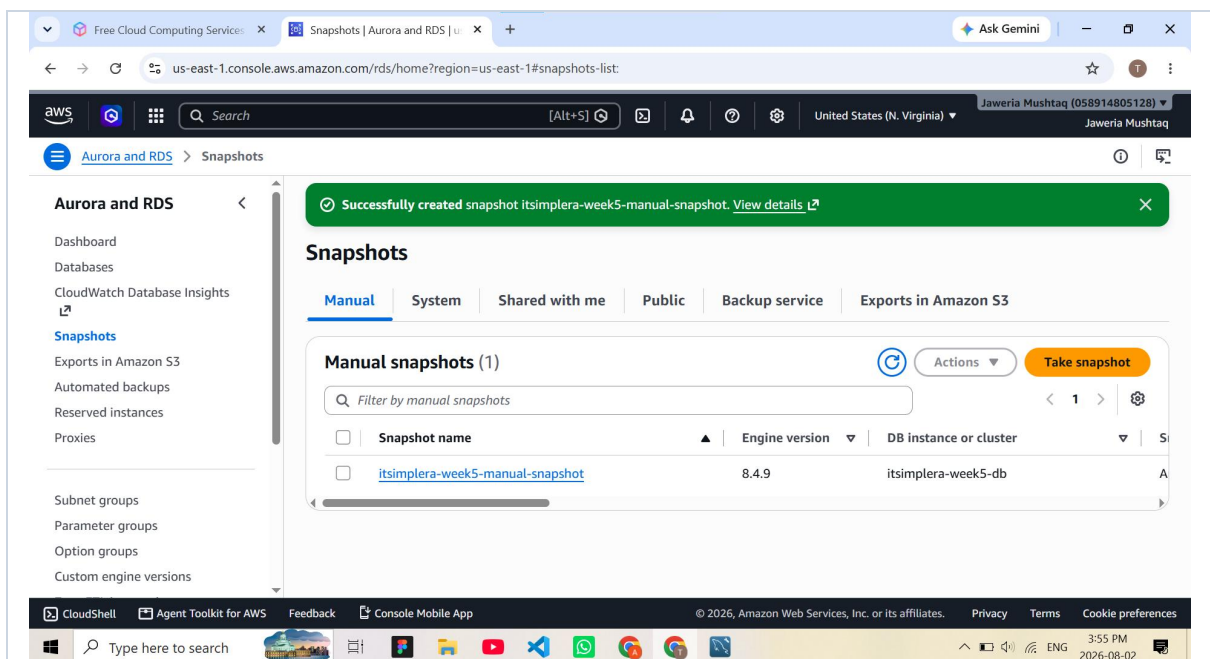


Fig 3.6d — Manual snapshot successfully created

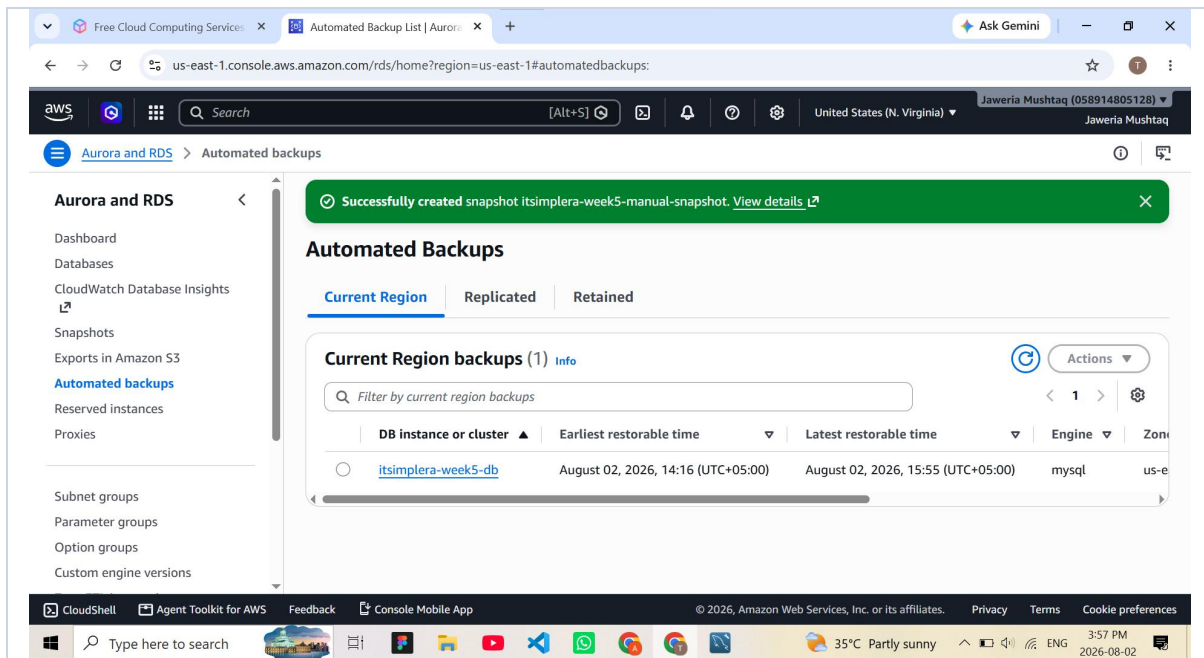


Fig 3.6e — Automated backups: current-region backup window for itsimplera-week5-db

4. Challenges Faced and Solutions Applied

- Confusion between "Easy create" and "Full configuration": the RDS creation wizard defaulted to Easy create, which does not expose the Public access setting needed for MySQL Workbench. Solution: manually switched to Full configuration before proceeding.
- Master password rejected ("The Master password is invalid"): the initial password contained the '@' symbol, which RDS master passwords do not allow. Solution: the password was changed to remove the disallowed character, after which it was accepted as "Strong."
- Accidental navigation away from the Create Database page before submission, resetting the form. Solution: the database creation steps were simply repeated from the start with no other impact, since nothing had been created yet.
- MySQL Workbench displayed a "Connection Warning" about server version 8.4.9 being newer than the versions Workbench is officially tested for (5.6–8.0). Solution: selected "Continue Anyway," and the connection completed successfully with no functional issues.

5. Learning Outcomes

- Understood the fundamentals of relational databases and how they differ from NoSQL databases such as DynamoDB.
- Learned to deploy and configure a managed MySQL database using Amazon RDS.
- Configured security group rules to allow secure remote connections to an RDS instance.
- Connected to an RDS MySQL instance using MySQL Workbench and verified the connection.
- Created tables, inserted records, and retrieved data using SQL (CREATE, INSERT, SELECT).
- Learned how to create manual snapshots and explored RDS backup and recovery options.

6. Conclusion

This assignment provided hands-on experience with AWS database services, focusing on Amazon RDS as a fully managed relational database platform. By launching a MySQL instance, connecting through MySQL Workbench, creating and querying tables with SQL, and performing a manual backup, practical understanding of relational database deployment, management, and recovery was achieved. This forms a strong foundation for working with both relational and NoSQL database services in future weeks.

7. RDS vs DynamoDB — Quick Comparison

Aspect	Amazon RDS (MySQL)	Amazon DynamoDB
Data model	Relational (tables, fixed schema)	Key-value / document (flexible schema)
Query language	SQL	API-based (GetItem, Query, Scan)
Scaling	Vertical (instance size) + read replicas	Automatic, virtually unlimited horizontal scaling
Best use case	Structured data with relationships (e.g., orders, users)	High-scale, low-latency apps (e.g., sessions, leaderboards)
Backup	Automated backups + manual snapshots	Point-in-time recovery + on-demand backups

8. RDS Endpoint / Connection Summary

```
Endpoint: itsimplera-week5-db.c6je2aok8i1t.us-east-1.rds.amazonaws.com
Port: 3306
Engine: MySQL 8.4.9
Master username: admin
Database name: itsimplera_week5
```